

Main is chapter ko section-by-section Hinglish mein unpack karunga, aur saath hi har feature ka actual object/prototype mental model aur practical use samjhaunga. Sirf aapke diye content par based rahunga. # Chapter 3: Classy Objects, Detailed Hinglish Explanation

Is chapter ka main purpose JavaScript ke class system ko samjhana hai, lekin sirf syntax ke level par nahi. Author baar-baar remind karta hai ki class ke neeche actual mechanism ab bhi **objects aur [[Prototype]] linking** hi hai.

Important: Chapter abhi “Work in progress” hai. Isliye kuch sections incomplete hain, jaise **Extending Expressions**.

1. JavaScript mein class ka basic idea

Class-oriented design mein hum generally:

1. Ek **type/category** define karte hain, jise class bolte hain.
2. Us class mein relevant data define karte hain.
3. Us data par kaam karne wale methods define karte hain.
4. Class se individual objects, yani instances, create karte hain.
5. Classes ke beech inheritance relationship bana sakte hain.

Example:

```
1  class User {
2      constructor(name) {
3          this.name = name;
4      }
5
6      greet() {
7          console.log(`Hello ${this.name}`);
8      }
9  }
10
11  const user1 = new User("Ruth");
12  const user2 = new User("Alex");
```

Yahan:

- User ek class hai.
 - user1 aur user2 independent instances hain.
 - name instance ka data hai.
 - greet() behavior/method hai.
-

ES6 se pehle classes kaise banti thi?

ES6, yani 2015 se pehle JavaScript mein class keyword nahi tha.

Developers plain constructor functions aur prototypes use karke class-like design banate the:

```
1  function User(name) {  
2      this.name = name;  
3  }  
4  
5  User.prototype.greet = function greet() {  
6      console.log(`Hello ${this.name}`);  
7  };
```

ES6 mein isi design ko express karne ke liye cleaner syntax aaya:

```
1  class User {  
2      constructor(name) {  
3          this.name = name;  
4      }  
5  
6      greet() {  
7          console.log(`Hello ${this.name}`);  
8      }  
9  }
```

Starting mein class ko mostly **syntactic sugar** maana gaya, kyunki internally prototype system hi use ho raha tha.

Lekin modern JavaScript mein classes ke paas kaafi dedicated features hain:

- public fields
- private fields
- private methods
- static properties
- static methods
- static initialization blocks
- extends
- super
- new.target

Isliye modern class sirf prettier prototype syntax nahi reh gaya. Phir bhi internal foundation ab bhi [[Prototype]] hi hai.

2. Classes kab use karni chahiye?

Class-orientation ek **design pattern** hai, universal solution nahi.

Har object ke liye class banana zaroori nahi hai.

“X is a Y” relationship

Class design tab naturally fit hota hai jab entities ke beech “**is-a**” relationship ho.

Example:

- Computer **is an** electronic device.
- Electronic device **is an** electrical device.

Conceptually:

```
1  class Electrical {
2      // common electrical behavior
3  }
4
5  class Electronic extends Electrical {
6      // additional electronic behavior
7  }
8
9  class Computer extends Electronic {
10     // computer-specific behavior
11 }
```

Yahan Electronic, Electrical ki properties aur behavior inherit kar sakta hai. Uske baad Computer, Electronic ka behavior inherit kar sakta hai.

Practical rule

Aapko classes tab useful lag sakti hain jab:

- same type ke multiple objects create karne hain
- un sab objects ke paas similar data aur behavior hoga
- shared methods chahiye
- inheritance meaningful hai
- subclasses base behavior ko extend ya override karenge

Example:

```
1  const user1 = new User("A");
2  const user2 = new User("B");
```

```
3  const user3 = new User("C");
```

Yahan multiple instances hain, isliye class useful ho sakti hai.

Agar poore program mein sirf ek fixed object chahiye:

```
1  const configuration = {  
2      theme: "dark",  
3      language: "en"  
4  };
```

To uske liye class unnecessary ho sakti hai.

3. Timesheet example: Encapsulation, inheritance aur composition

Author ne apne early software project ka example diya:

```
1  Timesheet  
2    └─ Week  
3      └─ Day  
4        └─ Task
```

Possible classes:

```
1  class Timesheet {}  
2  class Week {}  
3  class Day {}  
4  class Task {}
```

Har level par totalTime() method tha.

- Task.totalTime() apna time return karta.
- Day.totalTime() saare tasks ka time add karta.
- Week.totalTime() saare days ka time add karta.
- Timesheet.totalTime() saare weeks ka time add karta.

Encapsulation

Har entity apne internal details ko control karti hai.

For example, Timesheet ko ye jaanne ki zaroorat nahi ki ek individual task ka time exactly kaise calculate ho raha hai. Timesheet sirf week.totalTime() call karta hai.

```
1  class Timesheet {  
2      constructor(weeks) {  
3          this.weeks = weeks;
```

```

4     }
5
6     totalTime() {
7         return this.weeks.reduce(
8             (total, week) => total + week.totalTime(),
9             0
10        );
11    }
12 }

```

Isse internal implementation hide ho jati hai aur clean external interface milta hai.

Inheritance

Author ke design mein classes common base class ko extend karti thi, jisme common totalTime() behavior defined tha.

Inheritance ka idea:

```

1  class TimeCollection {
2      totalTime() {
3          // common summation logic
4      }
5  }
6
7  class Week extends TimeCollection {}
8  class Day extends TimeCollection {}

```

Composition

Composition ka meaning hai ek entity ke andar doosri entities rakhna.

- Timesheet is composed of weeks.
- Week is composed of days.
- Day is composed of tasks.

```

1  class Week {
2      constructor(days) {
3          this.days = days;
4      }
5  }

```

Important difference:

- Inheritance: Week is a TimeCollection
- Composition: Week has Days

4. Class declaration aur class expression

Class declaration

```
1  class Point2d {  
2      // ...  
3  }
```

Yahan Point2d ek class declaration hai.

Named class expression

```
1  const pointClass = class Point2d {  
2      // ...  
3  };
```

Class ka internal naam Point2d hai, lekin expression ko pointClass variable mein store kiya gaya hai.

Anonymous class expression

```
1  const anotherClass = class {  
2      // ...  
3  };
```

Yahan class expression ka explicitly diya gaya naam nahi hai.

5. Class methods ka syntax

```
1  class Point2d {  
2      setX(x) {  
3          // ...  
4      }  
5  
6      setY(y) {  
7          // ...  
8      }  
9  }
```

Class body mein method define karte waqt:

- function keyword nahi lagta
- methods ke beech comma nahi lagta

- normally semicolon bhi nahi lagta

Wrong:

```
1 class Point2d {  
2     function setX(x) {}  
3 }
```

Correct:

```
1 class Point2d {  
2     setX(x) {}  
3 }
```

Classes automatically strict mode mein hoti hain

Class body ke andar ka code strict mode mein execute hota hai, chahe file mein "use strict" likha ho ya nahi.

```
1 class Example {  
2     method() {  
3         // strict-mode rules apply here  
4     }  
5 }
```

Iska impact especially this ke behavior par padta hai.

6. Constructor kya hai?

constructor() ek special method hai jo har baar run hota hai jab class se new object banaya jata hai.

```
1 class Point2d {  
2     constructor() {  
3         console.log("New instance created");  
4     }  
5 }  
6  
7 const point = new Point2d();
```

Output:

```
1 New instance created
```

Agar constructor explicitly define nahi kiya, JavaScript default constructor assume karta hai.

Class technically ek special function hai

```
1 typeof Point2d;
```

Result:

```
1 "function"
```

Lekin class constructor ko regular function ki tarah call nahi kar sakte:

```
1 Point2d();
```

Result:

```
1 TypeError: Class constructor Point2d cannot be invoked without 'new'
```

Ye bhi allowed nahi:

```
1 Point2d.call({});
```

Class ko instantiate karne ke liye new use karna mandatory hai:

```
1 const point = new Point2d();
```

7. Har instance independent object hota hai

```
1 const one = new Point2d();
2 const two = new Point2d();
3 const three = new Point2d();
```

Ye teenon different objects hain.

```
1 one === two;
```

Result:

```
1 false
```

Agar property sirf one par add karein:

```
1 one.value = 42;
```

To:

```
1 one.value;    // 42
2 two.value;    // undefined
3 three.value;  // undefined
```

Har instance ke apne instance properties hote hain.

Lekin tino objects ka internal [[Prototype]] broadly same object ko point karega:

```
1 Point2d.prototype
```

Mental model:

```
1 one   _____
2 two   _____> Point2d.prototype
3 three _____
```

8. Class methods actually kahan store hote hain?

Consider:

```
1 class Point2d {
2     setX(x) {
3         console.log(`Setting x to: ${x}`);
4     }
5 }
6
7 const point = new Point2d();
8
9 point.setX(3);
```

Aisa lagta hai setX directly point ke andar hai. Lekin actually method yahan store hota hai:

```
1 Point2d.prototype.setX
```

Check:

```
1 Object.hasOwn(point, "setX");
2 // false
3
4 Object.hasOwn(Point2d.prototype, "setX");
5 // true
```

Jab hum likhte hain:

```
1 point.setX(3);
```

JavaScript roughly ye process follow karta hai:

1. point par setX search karo.
2. Wahan nahi mila.
3. point ke [[Prototype]] par jao.
4. Point2d.prototype par setX mil gaya.
5. Method execute karo.

Isliye methods har instance ke liye duplicate nahi hote. Ek method prototype par hota hai aur saare instances usse share karte hain.

Instance se method access karna chahiye

```
1 point.setX(3);
```

Ye proper use hai.

```
1 Point2d.setX(3);
```

Ye kaam nahi karega, kyunki setX static method nahi hai.

Technically ye reference exist karta hai:

```
1 Point2d.prototype.setX
```

Lekin normal class-oriented code mein instance method ko instance ke through invoke karna chahiye.

9. Class instance mein this

Class constructor aur instance methods ke andar this generally current instance ko refer karta hai.

```
1 class Point2d {  
2     constructor(x, y) {  
3         this.x = x;  
4         this.y = y;  
5     }  
6  
7     toString() {  
8         console.log(`(${this.x},${this.y})`);  
9     }  
10 }  
11  
12 const point = new Point2d(3, 4);  
13  
14 point.toString();
```

Output:

```
1 (3,4)
```

Jab constructor run hua:

```
1 this.x = x;  
2 this.y = y;
```

Yahan this newly created object tha.

Conceptually:

```
1 point.x = 3;
2 point.y = 4;
```

Jab call hua:

```
1 point.toString();
```

Method ke andar this === point hai.

Members aur methods

```
1 class Point2d {
2     constructor(x, y) {
3         this.x = x;
4         this.y = y;
5     }
6
7     toString() {
8         return `${this.x},${this.y}`;
9     }
10 }
```

- x aur y: members/data properties
 - toString(): method/behavior
-

10. Public fields

Constructor ke andar property add karne ke bajay class body mein declaratively fields define kar sakte hain:

```
1 class Point2d {
2     x = 0;
3     y = 0;
4
5     constructor(x, y) {
6         this.x = x;
7         this.y = y;
8     }
9 }
```

Har new instance ko apna x aur y milega.

```
1 const point1 = new Point2d(3, 4);
2 const point2 = new Point2d(10, 20);
```

point1.x aur point2.x independent properties hain.

Field without initialization

```
1 class Point2d {  
2     x;  
3     y;  
4 }
```

Default value:

```
1 undefined
```

Usually field ko class body ya constructor mein initialize karna chahiye.

Fields ek doosre ko access kar sakte hain

```
1 class Point3d {  
2     x;  
3     y = 4;  
4     z = this.y * 5;  
5 }
```

New object par:

```
1 y === 4  
2 z === 20
```

Field initializers order mein execute hote hain. z initialize hote waqt y already initialize ho chuka hai.

Public field ko constructor ke top jaisa samajh sakte hain

```
1 class Point2d {  
2     x = 0;  
3     y = 0;  
4 }
```

Rough mental model:

```
1 class Point2d {  
2     constructor() {  
3         this.x = 0;  
4         this.y = 0;  
5     }  
6 }
```

Lekin derived classes mein initialization order thoda different hota hai. Wahan fields super() ke baad initialize hote hain.

Computed field names

```
1  const coordName = "x";
2
3  class Point2d {
4      [coordName.toUpperCase()] = 42;
5  }
```

coordName.toUpperCase() gives "X".

Isliye instance par:

```
1  point.X; // 42
```

11. Arrow function ko instance field banana kyun problematic hai?

Chapter ka author is pattern ko anti-pattern maanta hai:

```
1  class Point2d {
2      x = null;
3      y = null;
4
5      getDoubleX = () => this.x * 2;
6
7      constructor(x, y) {
8          this.x = x;
9          this.y = y;
10     }
11 }
```

Iska benefit ye hai ki arrow function lexical this capture karta hai. Isliye method alag pass karne par bhi this lose nahi hoga.

Lekin problem ye hai ki:

```
1  getDoubleX = () => this.x * 2;
```

ek prototype method nahi ban raha. Ye har instance ke andar naya function create kar raha hai.

Equivalent behavior:

```
1 class Point2d {
2     constructor(x, y) {
3         this.getDoubleX = () => this.x * 2;
4         this.x = x;
5         this.y = y;
6     }
7 }
```

Check:

```
1 Object.hasOwn(point, "getDoubleX");
2 // true
```

Normal method ke case mein:

```
1 class Point2d {
2     getDoubleX() {
3         return this.x * 2;
4     }
5 }
```

Check:

```
1 Object.hasOwn(point, "getDoubleX");
2 // false
3
4 Object.hasOwn(Point2d.prototype, "getDoubleX");
5 // true
```

Difference

Normal method:

```
1 instance 1 ┌
2 instance 2 ┤──> one shared prototype method
3 instance 3 └
```

Arrow function field:

```
1 instance 1 ---> its own function
2 instance 2 ---> its own function
3 instance 3 ---> its own function
```

Author ke objections:

1. Har instance ke liye new function create hota hai.
2. Memory aur performance waste ho sakti hai.
3. Prototype-based method sharing ka benefit khatam ho jata hai.

4. Dynamic this ko avoid karke class design ke important feature ko neutralize kar dete hain.

Author ye nahi bol raha ki class ke andar arrow function kabhi use nahi karna. Wo specifically bol raha hai ki **instance method ki jagah arrow function field ko habit ke roop mein use mat karo.**

Preferred:

```
1 class Point2d {
2     getDoubleX() {
3         return this.x * 2;
4     }
5 }
```

12. Class extension aur inheritance

extends se class inheritance relationship define hota hai.

```
1 class Point2d {
2     x = 3;
3     y = 4;
4
5     getX() {
6         return this.x;
7     }
8 }
9
10 class Point3d extends Point2d {
11     x = 21;
12     y = 10;
13     z = 5;
14
15     printDoubleX() {
16         console.log(`double x: ${this.getX() * 2}`);
17     }
18 }
```

Yahan:

- Point2d: base class / parent class
- Point3d: derived class / child class / subclass

Point3d ko Point2d ka getX() method available hai.

```
1  const anotherPoint = new Point3d();
2
3  anotherPoint.getX(); // 21
```

Inherited getX() ye run karta hai:

```
1  return this.x;
```

this yahan anotherPoint hai, aur anotherPoint.x ki value 21 hai.

Important point:

Inherited method base class mein defined ho sakta hai, lekin uska this us instance ko refer karega jis par method call hua.

13. Method overriding

Subclass inherited method ko redefine kar sakti hai.

```
1  class Point2d {
2      x = 3;
3
4      getX() {
5          return this.x;
6      }
7  }
8
9  class Point3d extends Point2d {
10     x = 21;
11
12     getX() {
13         return this.x * 2;
14     }
15 }
```

Now:

```
1  const point = new Point3d();
2
3  point.getX(); // 42
```

Point3d ka getX() base class ke getX() ko override karta hai.

Base class ka method delete nahi hua. Wo ab bhi Point2d.prototype par exist karta hai.

14. super se parent method access karna

Agar subclass ne method override kar diya hai, phir bhi parent version call karna ho to super use karein:

```
1  class Point2d {
2      getX() {
3          return this.x;
4      }
5  }
6
7  class Point3d extends Point2d {
8      getX() {
9          return this.x * 2;
10     }
11
12     printX() {
13         console.log(`x: ${super.getX()}`);
14     }
15 }
```

Yahan:

```
1  super.getX()
```

Point2d.prototype.getX() ko access karta hai.

Lekin parent method ke andar this current Point3d instance hi hota hai.

Isliye agar this.x === 21:

```
1  super.getX(); // 21
2  this.getX();  // 42
```

Method polymorphism

Same method name inheritance hierarchy ke different levels par different behavior show kar sakta hai.

```
1  this.getX()
```

Current object se normal method resolution karega.

```
1  super.getX()
```

Parent class implementation ko use karega.

Isi capability ko **method polymorphism** bola gaya hai.

15. Subclass constructor aur super()

Agar subclass mein explicit constructor define karte hain, to parent constructor manually call karna hota hai:

```
1  class Point2d {
2      constructor(x, y) {
3          this.x = x;
4          this.y = y;
5      }
6  }
7
8  class Point3d extends Point2d {
9      constructor(x, y, z) {
10         super(x, y);
11         this.z = z;
12     }
13 }
```

super(x, y) yahan Point2d constructor ko call karta hai.

this se pehle super()

Derived class constructor mein this use karne se pehle super() call karna mandatory hai.

Wrong:

```
1  class Point3d extends Point2d {
2      constructor(x, y, z) {
3          this.z = z;
4          super(x, y);
5      }
6  }
```

Ye runtime error dega.

Correct:

```
1  class Point3d extends Point2d {
2      constructor(x, y, z) {
3          super(x, y);
4          this.z = z;
5      }
6  }
```

Agar subclass constructor define hi nahi karte, JavaScript automatic default constructor provide karta hai jo arguments parent ko forward karta hai.

Conceptually:

```
1 constructor(...args) {  
2     super(...args);  
3 }
```

16. Subclass field initialization order

Derived class mein fields constructor ke bilkul top par initialize nahi hote.

Example:

```
1 class Point3d extends Point2d {  
2     z = console.log("Initializing z");  
3  
4     constructor(x, y, z) {  
5         console.log("Point3d constructor started");  
6  
7         super(x, y);  
8  
9         console.log("Setting z");  
10        this.z = z;  
11    }  
12 }
```

Execution order:

```
1 1. Point3d constructor started  
2 2. Point2d constructor runs  
3 3. Point3d fields initialize  
4 4. Remaining Point3d constructor code runs
```

Yani:

```
1 super(x, y);
```

ke turant baad subclass fields initialize hote hain, phir constructor ki next statement chalti hai.

Mental model:

```
1 constructor(x, y, z) {  
2     console.log("Point3d constructor started");
```

```
3
4     super(x, y);
5
6     this.z = console.log("Initializing z");
7
8     console.log("Setting z");
9     this.z = z;
10 }
```

17. new.target

`new.target` batata hai ki object ko originally kis constructor/class ke through instantiate kiya gaya.

```
1 class Point2d {
2     constructor() {
3         console.log(new.target);
4     }
5 }
```

Direct creation:

```
1 new Point2d();
```

Yahan:

```
1 new.target === Point2d
```

Subclass creation:

```
1 class Point3d extends Point2d {}
2
3 new Point3d();
```

Jab `Point3d` ke through parent constructor run hoga, parent constructor ke andar bhi:

```
1 new.target === Point3d
```

hoga.

Ye detect karne mein useful hai ki base class directly instantiate hui ya subclass ke through.

18. instanceof

`instanceof` check karta hai ki kisi class ka `.prototype` object given object ki prototype chain mein present hai ya nahi.

```
1 class Point2d {}
2 class Point3d extends Point2d {}
3
4 const point = new Point2d();
5 const anotherPoint = new Point3d();
```

Results:

```
1 point instanceof Point2d; // true
2 point instanceof Point3d; // false
3
4 anotherPoint instanceof Point3d; // true
5 anotherPoint instanceof Point2d; // true
```

anotherPoint instanceof Point2d true isliye hai kyunki prototype chain approximately:

```
1 anotherPoint
2   ↓
3 Point3d.prototype
4   ↓
5 Point2d.prototype
6   ↓
7 Object.prototype
8   ↓
9 null
```

instanceof poori chain traverse karta hai.

19. isPrototypeOf()

Same relationship ko direct prototype perspective se check kar sakte hain:

```
1 Point2d.prototype.isPrototypeOf(anotherPoint);
2 // true
3
4 Point3d.prototype.isPrototypeOf(anotherPoint);
5 // true
```

Iska direct meaning:

Kya ye prototype object, anotherPoint ki prototype chain mein hai?

instanceof se relation:

```
1 anotherPoint instanceof Point2d
```

broadly yahi check karta hai:

```
1 Point2d.prototype.isPrototypeOf(anotherPoint)
```

20. Direct class check using constructor

Agar dekhna ho ki object directly kis class se bana hai:

```
1 point.constructor === Point2d;
2 // true
3
4 anotherPoint.constructor === Point3d;
5 // true
```

Lekin constructor property usually instance ki own property nahi hoti.

```
1 Object.hasOwn(anotherPoint, "constructor");
2 // false
```

Ye prototype par milti hai:

```
1 Point3d.prototype.constructor === Point3d;
2 // true
```

Isliye anotherPoint.constructor access prototype chain ke through resolve hota hai.

21. Inheritance copying nahi, sharing hai

Ye chapter ka sabse important mental model hai.

Jab Point3d extends Point2d likha jata hai, methods copy hokar har subclass aur instance mein nahi aate.

Instead, objects prototype links ke through connected hote hain.

```
1 class Point3d extends Point2d {
2     toString() {
3         return `${this.x},${this.y},${this.z}`;
4     }
5 }
6
7 const point = new Point3d(3, 4, 5);
```

Instance ki own properties:

```
1 Object.hasOwn(point, "x"); // true
2 Object.hasOwn(point, "y"); // true
```

```
3 Object.hasOwn(point, "z"); // true
4 Object.hasOwn(point, "toString"); // false
```

Method yahan hai:

```
1 Object.hasOwn(Point3d.prototype, "toString");
2 // true
```

point ko method prototype linkage se milta hai.

Correct mental model:

```
1 Methods copied down nahi hote.
2 Methods apni jagah par rehte hain.
3 Objects unhe prototype chain ke through share karte hain.
```

22. Static properties aur static methods

Ab tak class-related data teen locations par ho sakta hai:

1. Instance properties

```
1 point.x
```

Har instance ka separate data.

2. Prototype methods

```
1 Point2d.prototype.toString
```

Saare instances ke beech shared method.

3. Constructor/class object par statics

```
1 Point2d.origin
2 Point2d.distance()
```

Ye instance se nahi, directly class se related hote hain.

Static example

```
1 class Point2d {
2     static origin = new Point2d(0, 0);
3
4     static distance(point1, point2) {
5         return Math.sqrt(
6             ((point2.x - point1.x) ** 2) +
7             ((point2.y - point1.y) ** 2)
9     }
10 }
```

```
8         );  
9     }  
10  
11     constructor(x, y) {  
12         this.x = x;  
13         this.y = y;  
14     }  
15 }
```

Usage:

```
1 Point2d.origin;  
2 Point2d.distance(point1, point2);
```

Not:

```
1 point1.distance(point1, point2);
```

Kyunki distance instance method nahi, static function hai.

Static kab useful hai?

Static tab use karein jab behavior:

- class concept se related ho
- kisi particular instance ke this par depend na karta ho
- utility operation ho
- class-level constant/property ho

Example:

```
1 Point2d.distance(a, b)
```

Distance dono points ko input ke roop mein leta hai. Ye kisi ek point instance ka behavior hona zaroori nahi.

23. Static initializer mein this

Static context mein this instance ko nahi, class constructor ko refer karta hai.

```
1 class Point2d {  
2     static originX = 0;  
3     static originY = 0;  
4     static origin = new this(this.originX, this.originY);  
5 }
```

Yahan static initializer mein:

```
1 this == Point2d
```

Author new this(...) recommend nahi karta, kyunki explicit class name zyada readable hai:

```
1 static origin = new Point2d(  
2     this.originX,  
3     this.originY  
4 );
```

Static initialization kab hoti hai?

Instance field initialization tab hoti hai jab new se object banega.

```
1 class Example {  
2     value = calculateValue();  
3 }
```

calculateValue() har instance creation par chal sakta hai.

Lekin static initialization class define hote hi immediately run hoti hai:

```
1 class Example {  
2     static value = calculateValue();  
3 }
```

Class definition evaluate hote hi calculateValue() run hoga.

Static fields order mein initialize hote hain.

24. Static initialization blocks

ES2022 se static {} block available hai.

```
1 class Point2d {  
2     static origin = new Point2d(0, 0);  
3  
4     static distance(point1, point2) {  
5         return Math.sqrt(  
6             ((point2.x - point1.x) ** 2) +  
7             ((point2.y - point1.y) ** 2)  
8         );  
9     }  
10 }
```

```

11     static {
12         const outerPoint = new Point2d(6, 8);
13
14         this.maxDistance = this.distance(
15             this.origin,
16             outerPoint
17         );
18     }
19 }

```

Static block class definition ke time execute hota hai.

Result:

```

1 Point2d.maxDistance; // 10

```

Static block tab useful hai jab initialization simple expression se zyada complex ho:

- temporary variables
- loops
- conditions
- try...catch
- multiple related assignments

Static block ke local variables bahar accessible nahi hote:

```

1 outerPoint; // ReferenceError

```

25. Static inheritance

Statics bhi subclass ko inherited mil sakte hain.

```

1 class Point2d {
2     static maxDistance = 10;
3 }
4
5 class Point3d extends Point2d {}
6
7 Point3d.maxDistance; // 10

```

Ye bhi copying nahi hai.

Constructor functions ke beech prototype linkage banta hai:

```

1 Point3d constructor
2   ↓ [[Prototype]]

```

```
3 Point2d constructor
4   ↓
5 Function.prototype
```

Isliye Point3d.maxDistance resolve hote waqt JavaScript Point2d tak delegate kar sakta hai.

Static overriding

```
1 class Point2d {
2     static distance(a, b) {
3         // 2D distance
4     }
5 }
6
7 class Point3d extends Point2d {
8     static distance(a, b) {
9         // 3D distance
10    }
11 }
```

Point3d.distance() subclass implementation use karega.

Parent static function chahiye to:

```
1 super.distance(a, b)
```

use kar sakte hain.

26. Private class fields aur methods

Default class fields aur methods public hote hain:

```
1 class User {
2     name;
3     greet() {}
4 }
```

Outside se access possible hai:

```
1 user.name;
2 user.greet();
```

Private fields aur methods ke liye # syntax use hota hai.

```
1 class Point2d {
2     #ID = null;
```

```
3
4     #assignID() {
5         this.#ID = Math.round(Math.random() * 1e9);
6     }
7 }
```

Ye class ke bahar accessible nahi:

```
1 point.#ID;
```

Ye normal undefined result nahi dega. Ye syntax error hai.

27. Privacy ki motivation

Privacy ka purpose sirf values ko mysterious banana nahi hai.

Main reasons:

1. Implementation details hide karna

Consumer ko sirf public interface use karna chahiye.

```
1 class BankAccount {
2     #balance = 0;
3
4     deposit(amount) {
5         if (amount > 0) {
6             this.#balance += amount;
7         }
8     }
9
10    getBalance() {
11        return this.#balance;
12    }
13 }
```

External code direct balance modify nahi kar sakta:

```
1 account.#balance = 999999;
```

2. Principle of Least Privilege

Sirf wahi data aur capability expose karo jo actually necessary hai.

3. Refactoring safety

Agar internal implementation public hai, doosra code us par depend kar sakta hai. Phir internal implementation change karna difficult ho jata hai.

Private implementation ko freely refactor kiya ja sakta hai, jab tak public interface same rahe.

28. JavaScript privates ki limitation

JavaScript private fields sirf **usi exact class** ke liye private hain jahan declare hue.

Subclass unhe access nahi kar sakti.

```
1  class Point2d {
2      #ID = 123;
3  }
4
5  class Point3d extends Point2d {
6      printID() {
7          console.log(this.#ID);
8      }
9  }
```

Ye class definition ke time syntax error dega.

Subclass ye bhi nahi kar sakti:

```
1  super.#ID
```

Traditional languages mein protected visibility hoti hai:

- outside code se hidden
- class ke andar accessible
- subclasses ke andar bhi accessible

JavaScript mein native protected visibility nahi hai.

Isliye decision difficult ho sakta hai:

- public rakho, to overexposed ho sakta hai
- private rakho, to subclass use nahi kar sakti

Author isi wajah se private features ko cautiously use karne ko keh raha hai.

29. Private fields ko declare karna mandatory hai

Public property ko dynamically add kar sakte hain:

```
1 class Example {  
2     constructor() {  
3         this.value = 10;  
4     }  
5 }
```

Lekin private field ko pehle class body mein declare karna compulsory hai.

Correct:

```
1 class Example {  
2     #value;  
3  
4     constructor() {  
5         this.#value = 10;  
6     }  
7 }
```

Incorrect:

```
1 class Example {  
2     constructor() {  
3         this.#value = 10;  
4     }  
5 }
```

Agar #value declared nahi hai, syntax error milega.

Private field ko reassign kar sakte hain:

```
1 this.#value = 20;
```

Lekin delete nahi kar sakte:

```
1 delete this.#value;
```

30. Private field subclass instance par exist kar sakta hai

Ye subtle point hai.

```
1 class Point2d {  
2     #ID;  
3 }
```

```

4     constructor() {
5         this.#ID = 123;
6     }
7
8     getID() {
9         return this.#ID;
10    }
11 }
12
13 class Point3d extends Point2d {
14     constructor() {
15         super();
16     }
17 }
18
19 const point = new Point3d();

```

super() parent constructor ko current Point3d instance par run karta hai. Isliye us point object ke private storage mein Point2d ka #ID set ho jata hai.

Inherited method access kar sakta hai:

```

1 point.getID(); // 123

```

Lekin Point3d ke apne methods directly #ID access nahi kar sakte.

Meaning:

- Private state subclass instance ke saath associated ho sakta hai.
- Lekin sirf declaring class ka code usse access kar sakta hai.

31. Private brand check using in

Agar check karna ho ki object mein specific class ka private field hai:

```

1 #ID in object

```

Example:

```

1 class Point2d {
2     #ID;
3
4     static isPoint2d(value) {
5         return #ID in value;
6     }

```

```
7 }
```

Usage:

```
1 Point2d.isPoint2d(new Point2d());
2 // true
3
4 Point2d.isPoint2d({});
5 // false
```

Ye check sirf us class ke andar likha ja sakta hai jo #ID declare karti hai.

Direct private field access:

```
1 object.#ID
```

agar invalid object par ho to exception produce karega.

Brand check:

```
1 #ID in object
```

safe boolean deta hai. Isliye try...catch ki zaroorat nahi.

32. Private members ki exfiltration

Private ka meaning ye nahi ki class ka code unhe bahar leak hi nahi kar sakta.

```
1 let leakedID;
2 let leakedFunction;
3
4 class Point2d {
5     #ID = 100;
6
7     #assignID() {
8         this.#ID = 200;
9     }
10
11     constructor() {
12         leakedID = this.#ID;
13         leakedFunction = this.#assignID;
14     }
15 }
```

Class ke andar private value ko public variable mein assign kar diya gaya. Ab wo bahar available hai.

```
1 leakedID; // 100
```

Private method function reference bhi bahar ja sakta hai:

```
1 leakedFunction;
```

Lekin private method ka correct branded instance hona zaroori hai:

```
1 leakedFunction.call(point);
```

kaam kar sakta hai.

```
1 leakedFunction.call({});
```

fail karega, kyunki plain object ke private storage mein declaring class ka #ID nahi hai.

Lesson:

Private syntax accidental outside access rokta hai, lekin class ke apne code ko private value leak karne se nahi rokta.

Isliye private callbacks expose karte waqt careful rehna chahiye.

33. Private static properties aur functions

Static members bhi private ho sakte hain:

```
1 class Point2d {
2     static #errorMessage = "Out of bounds.";
3
4     static #printError() {
5         console.log(`Error: ${this.#errorMessage}`);
6     }
7
8     constructor(x, y) {
9         if (x > 100 || y > 100) {
10             Point2d.#printError();
11         }
12     }
13 }
```

Outside se:

```
1 Point2d.#errorMessage;
2 Point2d.#printError();
```

allowed nahi hai.

Ye values instance se related nahi hain, isliye static hain. Aur external code ko inki zaroorat nahi, isliye private hain.

34. Private static + inheritance gotcha

Consider:

```
1 class Point2d {
2     static #errorMsg = "Out of bounds.";
3
4     static printError() {
5         console.log(this.#errorMsg);
6     }
7 }
8
9 class Point3d extends Point2d {}
```

Direct parent call:

```
1 Point2d.printError();
```

works because:

```
1 this === Point2d
```

Lekin:

```
1 Point3d.printError();
```

fail karega.

Why?

printError() function inherited hai, lekin call Point3d ke through hua, to function ke andar:

```
1 this === Point3d
```

#errorMsg Point2d ka private static field hai. Point3d ne us private field ko declare nahi kiya.

Isliye:

```
1 this.#errorMsg
```

fail ho jata hai.

Fix

Private static ko explicit class name se access karein:

```
1 class Point2d {
2     static #errorMsg = "Out of bounds.";
3
4     static printError() {
```

```
5         console.log(Point2d.#errorMsg);
6     }
7 }
```

Now both calls work:

```
1 Point2d.printError();
2 Point3d.printError();
```

Rule:

Agar public static method inherit ho sakta hai aur uske andar base class ka private static access ho raha hai, to this.#private ke bajay BaseClass.#private use karna safer hai.

35. Final Calendar example ka complete breakdown

Final example mein teen classes hain:

```
1 CalendarItem
2   └─ Reminder
3   └─ Meeting
```

CalendarItem base class hai.

Reminder aur Meeting subclasses hain.

35.1 CalendarItem as an abstract-style class

Author nahi chahta ki CalendarItem directly instantiate ho.

Allowed:

```
1 new Reminder(...);
2 new Meeting(...);
```

Disallowed:

```
1 new CalendarItem();
```

JavaScript mein built-in abstract class keyword nahi hai. Isliye author new.target se abstract-class-like behavior create karta hai.

```
1 constructor() {
2     if (new.target !== CalendarItem) {
3         // subclass se instantiate hua
4     }
5     else {
```

```
6         throw new Error(...);
7     }
8 }
```

Direct call:

```
1 new CalendarItem();
```

Yahan:

```
1 new.target === CalendarItem
```

Isliye error throw hoga.

Subclass:

```
1 new Reminder();
```

Base constructor ke andar:

```
1 new.target === Reminder
```

Isliye construction allowed hai.

35.2 static #UNSET

```
1 static #UNSET = Symbol("unset");
```

Ye ek private static unique sentinel value hai.

Iska purpose batana hai ki ID abhi set nahi hui.

null ya undefined ki jagah Symbol use karne ka benefit ye hai ki symbol unique hota hai.

```
1 #ID = CalendarItem.#UNSET;
```

Initial state:

```
1 ID currently unavailable
```

35.3 static #isUnset(v)

```
1 static #isUnset(v) {
2     return v === this.#UNSET;
3 }
```

Ye supplied value ko private #UNSET sentinel se compare karta hai.

Lekin earlier gotcha ko dekhte hue, inherited static context mein this.#UNSET problematic ho sakta hai. Is example mein ye private static method mainly explicit CalendarItem.#isUnset(...) se call ho raha hai, isliye this CalendarItem rahega.

35.4 Dynamic error properties

```
1 static #error(num) {  
2     return this[`ERROR_${num}`];  
3 }
```

Agar:

```
1 num === 100
```

to property access banega:

```
1 this.ERROR_100
```

35.5 Static initialization block

```
1 static {  
2     for (let [idx, msg] of [  
3         "ID is already set.",  
4         "ID is unset.",  
5         "Don't instantiate 'CalendarItem' directly.",  
6     ].entries()) {  
7         this[`ERROR_${(idx + 1) * 100}`] = msg;  
8     }  
9 }
```

Loop properties create karega:

```
1 CalendarItem.ERROR_100 = "ID is already set.";  
2 CalendarItem.ERROR_200 = "ID is unset.";  
3 CalendarItem.ERROR_300 =  
4     "Don't instantiate 'CalendarItem' directly.";
```

Yahan computed property names use hue:

```
1 this[`ERROR_${number}`]
```

35.6 isSameItem()

```
1 static isSameItem(item1, item2) {  
2     if (#ID in item1 && #ID in item2) {  
3         return item1.#ID === item2.#ID;  
4     }  
5  
6     return false;  
7 }
```

Pehle brand check:

```
1 #ID in item1
2 #ID in item2
```

Ye confirm karta hai ki dono objects CalendarItem private brand carry karte hain.

Uske baad IDs compare hoti hain:

```
1 item1.#ID === item2.#ID
```

Same object same ID dega:

```
1 CalendarItem.isSameItem(item, item);
2 // true
```

Different calendar items usually different random IDs denge:

```
1 CalendarItem.isSameItem(item1, item2);
2 // false
```

35.7 Private #ID

```
1 #ID = CalendarItem.#UNSET;
```

Har instance ko apna private ID storage milta hai.

Ye public nahi:

```
1 item.#ID;
```

allowed nahi.

35.8 Private #setID()

```
1 #setID(id) {
2     if (CalendarItem.#isUnset(this.#ID)) {
3         this.#ID = id;
4     }
5     else {
6         throw new Error(CalendarItem.#error(100));
7     }
8 }
```

ID sirf ek baar set ho sakti hai.

First call:

```
1 Current ID = UNSET
2 New ID assign ho jayegi
```

Second call:

```
1 Current ID already set
2 Error: ID is already set.
```

Ye internal invariant protect karta hai.

35.9 Public fields

```
1 description = null;
2 startDateTime = null;
```

Ye har instance ki public properties hain.

External code inhe access kar sakta hai:

```
1 item.description;
2 item.startDateTime;
```

35.10 Base constructor aur random ID

```
1 constructor() {
2     if (new.target !== CalendarItem) {
3         const id = Math.round(Math.random() * 1e9);
4         this.#setID(id);
5     }
6     else {
7         throw new Error(CalendarItem.#error(300));
8     }
9 }
```

Subclass instance hone par:

1. Random ID generate karo.
2. Private #setID() se assign karo.

Direct CalendarItem creation par error.

35.11 getID()

```
1 getID() {
2     if (!CalendarItem.#isUnset(this.#ID)) {
3         return this.#ID;
4     }
5     else {
6         throw new Error(CalendarItem.#error(200));
7     }
8 }
```

```
7     }  
8 }
```

Agar ID set hai to return karo.

Agar unset hai:

```
1 Error: ID is unset.
```

Private data ko controlled public method ke through expose kiya gaya hai.

35.12 getDateTimeStr()

```
1 getDateTimeStr() {  
2     if (this.startDateTime instanceof Date) {  
3         return this.startDateTime.toUTCString();  
4     }  
5 }
```

Pehle check:

```
1 this.startDateTime instanceof Date
```

Agar actual Date object hai to UTC string return hoti hai.

35.13 summary()

```
1 summary() {  
2     console.log(`(${  
3         this.getID()  
4     }) ${  
5         this.description  
6     } at ${  
7         this.getDateTimeStr()  
8     }`);  
9 }
```

Output approximately:

```
1 (123456789) Call my parents at Thu, 07 Jul 2022 11:00:00 GMT
```

Base class generic summary behavior provide karti hai.

36. Reminder class

```
1 class Reminder extends CalendarItem
```

Meaning: Reminder ek CalendarItem hai.

Private completion state

```
1 #complete = false;
```

Har Reminder ka private completion status hai.

External code direct access nahi kar sakta:

```
1 reminder.#complete;
```

Symbol.toStringTag

```
1 [Symbol.toStringTag] = "Reminder";
```

Ye object ka built-in string tag customize karta hai.

Isliye:

```
1 reminder.toString();
```

result:

```
1 [object Reminder]
```

Reminder constructor

```
1 constructor(description, startDateTime) {  
2     super();  
3  
4     this.description = description;  
5     this.startDateTime = startDateTime;  
6 }
```

Sequence:

1. super() calls CalendarItem constructor.
 2. Base constructor random private ID set karta hai.
 3. Base public fields initialize hote hain.
 4. Reminder ka private #complete aur public tag initialize hote hain.
 5. Remaining constructor code description aur start time set karta hai.
-

Completion methods

```
1  isComplete() {  
2      return !!this.#complete;  
3  }
```

Double negation value ko boolean mein normalize karti hai.

```
1  markComplete() {  
2      this.#complete = true;  
3  }
```

Controlled method private state change karta hai.

Overridden summary()

```
1  summary() {  
2      if (this.isComplete()) {  
3          console.log(`(${this.getID()}) Complete.`);  
4      }  
5      else {  
6          super.summary();  
7      }  
8  }
```

Agar reminder complete hai:

```
1  (ID) Complete.
```

Otherwise parent ka summary:

```
1  super.summary();
```

Ye overriding aur parent-method access dono demonstrate karta hai.

37. Meeting class

```
1  class Meeting extends CalendarItem
```

Meeting bhi CalendarItem hai.

Private method

```
1  #getEndTimeStr() {  
2      if (this.endTime instanceof Date) {  
3          return this.endTime.toUTCString();  
4      }
```

```
5 }
```

Ye sirf Meeting class ke andar call ho sakta hai.

Public endTime

```
1 endTime = null;
```

Ye public instance field hai.

Meeting constructor

```
1 constructor(  
2     description,  
3     startDateTime,  
4     endTime  
5 ) {  
6     super();  
7  
8     this.description = description;  
9     this.startDateTime = startDateTime;  
10    this.endTime = endTime;  
11 }
```

Base initialization ke baad meeting-specific end time assign hota hai.

Overridden getDateTimestr()

```
1 getDateTimestr() {  
2     return `${  
3         super.getDateTimestr()  
4     } - ${  
5         this.#getEndTimeStr()  
6     }`;   
7 }
```

Start time ke liye parent method:

```
1 super.getDateTimestr()
```

End time ke liye private method:

```
1 this.#getEndTimeStr()
```

Result:

```
1 start time - end time
```

Meeting ne base behavior ko completely discard nahi kiya. Usne parent behavior ko reuse karke extend kiya.

38. isSameItem() subclasses ke through kaise available hai?

Method base class par static hai:

```
1 CalendarItem.isSameItem
```

Lekin static inheritance ki wajah se ye bhi available hai:

```
1 Reminder.isSameItem  
2 Meeting.isSameItem
```

Example:

```
1 Reminder.isSameItem(callMyParents, callMyParents);  
2 // true  
3  
4 Meeting.isSameItem(callMyParents, interview);  
5 // false
```

Meeting.isSameItem() ka matlab ye nahi ki dono arguments Meeting hone chahiye. Inherited function simply CalendarItem private IDs compare kar raha hai.

39. Example mein semicolon warning

Code mein comments hain:

```
1 #complete = false; // no ASI, semicolon needed  
2  
3 endDateTime = null; // no ASI, semicolon needed
```

Reason next field computed name se start hota hai:

```
1 [Symbol.toStringTag] = "Reminder"
```

Agar previous field ke baad semicolon omit ho, parser dono lines ko accidentally ek combined expression ke roop mein interpret kar sakta hai.

Safe:

```
1 #complete = false;
```

```
2 [Symbol.toStringTag] = "Reminder";
```

Isi tarah:

```
1 endDateTime = null;  
2 [Symbol.toStringTag] = "Meeting";
```

Class fields ke saath semicolons consistently use karna parsing ambiguity se bachata hai.

40. CalendarItem constructor arguments kyun nahi leta?

Repeated code subclasses mein hai:

```
1 this.description = description;  
2 this.startDateTime = startDateTime;
```

Isse base constructor mein move kiya ja sakta tha:

```
1 constructor(description, startDateTime) {  
2     this.description = description;  
3     this.startDateTime = startDateTime;  
4 }
```

Phir subclass:

```
1 super(description, startDateTime);
```

Lekin author intentionally aisa nahi karta.

Uska reasoning:

- CalendarItem directly instantiate nahi honi chahiye.
- Ye abstract-style base class hai.
- Agar constructor signature CalendarItem(description, startDateTime) rakhenge, to lag sakta hai ki ise directly use karna intended hai.
- Isliye concrete subclasses apni required data initialize karti hain.

Ye ek design choice hai, language requirement nahi.

41. Three important storage locations

Classes samajhne ke liye ye distinction bahut important hai.

Instance properties

```
1 this.x = 10;
```

Location:

```
1 point.x
```

Har instance ke paas apni copy.

Check:

```
1 Object.hasOwnProperty(point, "x");  
2 // true
```

Prototype methods

```
1 getX() {  
2     return this.x;  
3 }
```

Location:

```
1 Point2d.prototype.getX
```

Instances ke beech shared.

Check:

```
1 Object.hasOwnProperty(point, "getX");  
2 // false  
3  
4 Object.hasOwnProperty(Point2d.prototype, "getX");  
5 // true
```

Static properties/functions

```
1 static origin = new Point2d(0, 0);  
2  
3 static distance(a, b) {}
```

Location:

```
1 Point2d.origin  
2 Point2d.distance
```

Ye constructor function object ke properties hain.

42. Complete prototype mental model

Given:

```

1  class Point2d {
2      static category = "point";
3
4      constructor(x, y) {
5          this.x = x;
6          this.y = y;
7      }
8
9      getX() {
10         return this.x;
11     }
12 }
13
14 class Point3d extends Point2d {
15     constructor(x, y, z) {
16         super(x, y);
17         this.z = z;
18     }
19
20     getZ() {
21         return this.z;
22     }
23 }
24
25 const point = new Point3d(1, 2, 3);

```

Instance-side chain:

```

1  point
2    own: x, y, z
3      ↓ [[Prototype]]
4  Point3d.prototype
5    own: constructor, getZ
6      ↓ [[Prototype]]
7  Point2d.prototype
8    own: constructor, getX
9      ↓ [[Prototype]]
10 Object.prototype
11     ↓
12 null

```

Static-side chain:

```
1 Point3d
2   own static properties
3   ↓ [[Prototype]]
4 Point2d
5   own: category
6   ↓ [[Prototype]]
7 Function.prototype
8   ↓
9 Object.prototype
10  ↓
11 null
```

Isi wajah se:

```
1 point.getX();
```

works through instance-side prototype chain.

Aur:

```
1 Point3d.category;
```

works through constructor-side prototype chain.

43. Chapter ka practical decision guide

Classes use karne ka strong case

Classes consider karein agar:

- same type ke multiple instances banenge
- instances shared methods use karenge
- entity ke data aur behavior ko saath organize karna hai
- meaningful is-a relationship hai
- inheritance/overriding actually useful hai
- polymorphism ki need hai

Example:

```
1 CalendarItem
2   └─ Reminder
3   └─ Meeting
```

Yahan inheritance reasonably meaningful hai.

Classes avoid karne ka case

Classes unnecessary ho sakti hain agar:

- sirf ek object chahiye
 - inheritance ki need nahi
 - simple functions aur objects enough hain
 - aap methods ko mostly arrow-function fields bana rahe ho
 - आपको fixed lexical contexts chahiye, dynamic this nahi
 - composition ya closures problem ko simpler solve kar sakte hain
-

44. Most important takeaways

1. class ke neeche prototypes hain

```
1 class syntax
2   ↓
3 prototype objects
4   ↓
5 [[Prototype]] links
```

2. Instances independent data rakhte hain

```
1 this.x
2 this.y
```

Har instance ki own properties hoti hain.

3. Instance methods prototype par shared hote hain

```
1 Point2d.prototype.getX
```

Har instance ke andar copy nahi hoti.

4. Inheritance copying nahi hai

Subclass aur instances prototype chain ke through behavior share karte hain.

5. this current invocation ke context par depend karta hai

```
1 point.getX();
```

mein this generally point hai.

6. super() parent constructor run karta hai

Derived constructor mein this use karne se pehle super() mandatory hai.

7. super.method() parent implementation access karta hai

Overridden method ke bawajood base method call kar sakte hain.

8. Static behavior class par hota hai

```
1 Point2d.distance()
```

instance par nahi.

9. Private fields # se define hote hain

```
1 #ID
2 #assignID()
```

Sirf declaring class unhe directly access kar sakti hai.

10. JavaScript mein protected nahi hai

Subclass base class ke private fields access nahi kar sakti.

11. Arrow function fields prototype methods nahi hote

```
1 method = () => {};
```

har instance mein separate function banata hai.

12. Class tab use karo jab pattern genuinely fit ho

Har object ko class banana good design nahi hota.

Final simplified summary

JavaScript classes ko samajhne ka best mental model ye hai:

```
1 Class = instance creation ka structure
2       + prototype par shared methods
3       + inheritance ke prototype links
4       + optional static behavior
5       + optional private state
```

Jab aap likhte hain:

```
1 class Child extends Parent {}
```

to JavaScript behaviors ko objects mein copy nahi kar raha. Wo prototype objects aur constructor functions ke beech links establish kar raha hai.

Isi linking ki wajah se:

- methods share hote hain

- extends work karta hai
- super work karta hai
- instanceof work karta hai
- static inheritance work karti hai
- method overriding aur polymorphism possible hote hain

Sabse important design lesson ye hai ki **classes JavaScript mein available hain, lekin har problem ke liye necessary nahi hain**. Classes tab sabse useful hoti hain jab multiple similar instances, meaningful inheritance, shared behavior aur polymorphism ki real need ho.